

Numerical Sieves in Haskell (Eratosthenes', Euler's)

Leif Nyland

The program implements semi-faithful functional implementations of Euler's and Eratosthenes' sieves, although the implementation of the Sieve of Eratosthenes does use mod instead of index jumps to mark values for elimination. Functional implementations of sieves can make use of infinite lists, so no code is required to handle the range of integers required to locate a specific prime number. Each sieve has a function for producing a list of the first n prime numbers, and a function for producing the nth prime number. The sieve program does not have a main, as I was more interested in the actual operations performed by the code instead of a user interface. I do not believe that a main function would have significantly made the program more useful.

There are 4 sections in the program, all labeled with comments. The top section has imports and helper functions. The next two sections contain the functions for my implementations of the sieves. Each function is labeled with a comment that describes its purpose. The last section is just an implementation of an unfaithful Sieve of Eratosthenes that I found in a paper about sieve implementations in functional languages, which is included to demonstrate the code compression that can be achieved in Haskell. Here is a list of the important functions:

- `listEratDiv :: Integer -> [Integer]`
 - Returns a list of the first n prime numbers, calculated by the Sieve of Eratosthenes using mod (division).
- `nthEratDiv :: Integer -> Integer`
 - Returns the nth prime number, calculated by the Sieve of Eratosthenes using mod (division).
- `listEulers :: Integer -> [Integer]`
 - Returns a list of the first n prime numbers, calculated by Euler's Sieve.
- `nthEulers :: Integer -> Integer`
 - Returns the nth prime number, calculated by Euler's Sieve.
- `primes :: [Integer]`
 - Returns an infinite list of primes, calculated by a more compressed version of the Sieve of Eratosthenes. This code comes from "Three Euler's Sieves and a Fast Prime Generator (Functional Pearl)" by Ivano Salvo, Agnese Pacifico.

I chose this problem because I was interested in the use of infinite lists and lazy evaluation to solve sieve problems. In an imperative language, the array of numbers is limited to a finite length. Since it is impossible to know if the desired prime is in a finite list of integers, it would be necessary to implement a search through/grow list style of implementation. On the other hand, Haskell allows the appearance of an infinite list, so no expand list operations are required. Although this is a nice benefit, functional sieve implementations quickly run into problems that drastically impact runtime efficiency. The inability to index lists in constant time

becomes a problem. Euler's Sieve is particularly tricky with common implementations of functional languages, as it is necessary to both write and read a list at the same time, and all of the writes are entirely based on the value in the cell. In an array, these writes can be constant time, but my current implementation uses a linear search, even though the list is in order. Runtime could be drastically improved with array style accesses, or even some sort of $O(\log n)$ search algorithm. Even though Euler's Sieve has several theoretical runtime improvements over the Sieve of Eratosthenes, my current implementation performs significantly slower for the same search.

For future iterations, I will look into the potential optimizations available for both functional and imperative implementations of numerical sieves. The poor performance of my implementations is a blatant problem, and one where slight improvements could have drastic effects. Adding a main could also be beneficial, but this program was not originally intended to be a complete product. The goal of this project is to play around with using functional definitions of numerical sieves, and a main function does not necessarily improve its ability to complete the goal.

I have two major takeaways from my final project on numerical sieves. First, coding a theoretical mathematical concept in Haskell can be completed far more easily than in an imperative language. My implementations of the two sieves are far more concise than they would be in an imperative language, and they are a more direct translation of the theoretical numerical sieve process. As shown by the following example from Ivano Salvo and Agnese Pacifico, my implementations could be simplified much further.

```
primes = filterPrime [2..]
  where filterPrime (p:xs) =
    p : filterPrime [x | x <- xs, x `mod` p /= 0]
```

Second, functional environments allow for calculation efficiencies in some areas, but they also have some limitations. Since most functional languages are limited to lists instead of arrays, some simple operations are not feasible when runtime is a concern. Because of the specific limitations of functional languages, I would love to spend more time learning specifically about runtime improvements for functional languages, as I can foresee fun uses for runtime efficient functional implementations of a problem.

Sources

Ivano Salvo, Agnese Pacifico. *Three Euler's Sieves and a Fast Prime Generator (Functional Pearl)*. <https://arxiv.org/pdf/1811.09840>